

# Indent Validation Service

Production Architecture, Integration & Data Flow

Version 1.1 (Draft)

## 1. Executive Summary

The Indent Validation Service is a stateless service that integrates with IPMS to evaluate procurement indents and return an adequacy score together with a detailed analysis. It is invoked when a user selects the “Check Adequacy” action on an indent in IPMS.

This document focuses on production integration: the APIs the service exposes and consumes, the database tables IPMS must create, the system architecture, and the runtime data flow. The internal analysis logic is summarised only briefly, as it is not the integration concern.

The service does not store data. It receives an indent identifier, gathers the indent's field data and document attachments, analyses them, and returns a structured response. IPMS persists the response - the score and the full detailed analysis - and manages all history and display.

**Advisory, not blocking.** The adequacy score is guidance, not a gate. Users may submit at any score. Both the user and the manager see the same detailed analysis.

The system has two subsystems:

- **Live Validation API** - handles each request, returns results, reads the standard from memory, never writes to the database.
- **Offline Standard Pipeline** - scheduled; rebuilds the standard from accumulated indents and publishes a new version to the database.

**LLM access.** All language-model calls are made through an authenticated GENAI API over an authenticated session.

## 2. API Contracts

This section defines every integration point. Each contract is described as a field table: the key name, its data type (string, number, boolean, list, or object), and its meaning.

### APIs required

The integration involves one API the service exposes and three it consumes:

| API                | Direction                       | Purpose                                           |
|--------------------|---------------------------------|---------------------------------------------------|
| Check-Adequacy API | IPMS to Service (exposed by us) | Receives the request, returns the scored analysis |
| Indent Field API   | Service to IPMS (consumed)      | Provides the indent's filled field data           |
| Document Store API | Service to Store (consumed)     | Provides the indent's attachments via OAuth       |
| GENAI API          | Service to GENAI (consumed)     | Performs extraction and analysis (authenticated)  |

### 2.1 Check-Adequacy API - request

POST /check-adequacy - body is an object:

| Key       | Type   | Description                          |
|-----------|--------|--------------------------------------|
| indent_id | string | Identifier of the indent to validate |
| domain    | string | Indent domain (selects the standard) |

| Key          | Type   | Description                                      |
|--------------|--------|--------------------------------------------------|
| callback_url | string | Where the result is delivered when ready (async) |

Immediate response (before analysis completes) - an object:

| Key         | Type   | Description                        |
|-------------|--------|------------------------------------|
| analysis_id | string | Identifier for this analysis run   |
| status      | string | Processing state (e.g. processing) |

## 2.2 Check-Adequacy API - result

Delivered when analysis completes. This is the full payload IPMS persists - the score and the complete detailed analysis. An object with these keys:

| Key                      | Type            | Description                                  |
|--------------------------|-----------------|----------------------------------------------|
| analysis_id              | string          | Identifier for this analysis run             |
| indent_id                | string          | The indent that was analysed                 |
| domain                   | string          | Indent domain                                |
| analysis_timestamp       | string          | When the analysis ran (ISO timestamp)        |
| standard_version         | string          | Which standard version was used              |
| score                    | number          | Overall adequacy score, 0-100                |
| grade                    | string          | Strong / Adequate / Needs Improvement / Weak |
| score_breakdown          | object          | Points per category                          |
| findings                 | list of objects | Each: category, status, title, detail        |
| recommendations          | list of strings | Actions to improve the indent                |
| missing_documents        | list of strings | Required document types not found            |
| interrelationship_issues | list of strings | Cross-document inconsistencies               |
| document_inventory       | list of objects | Each: file, classified_as, genuine (boolean) |

**Detailed analysis stored with the score.** IPMS persists the entire detailed analysis alongside the score, so the user and manager always see the full reasoning, not just the number.

## 2.3 Indent Field API - consumed

Returns an object with indent\_id and fields, where fields is a list of objects - one per filled field:

| Key (per field object) | Type    | Description                                    |
|------------------------|---------|------------------------------------------------|
| question               | string  | The field label shown to the user              |
| answer                 | string  | The value selected or entered                  |
| required               | boolean | Whether the field was mandatory (the * fields) |

**Dynamic fields.** Fields are conditional - selecting one option unlocks others - so the key set varies between indents. The service consumes whatever question / answer / required entries are returned; new or renamed fields do not break it.

## 2.4 Document Store API - consumed

Returns the indent's attachments over an authenticated (OAuth) connection: a list of files (binary content plus original filename). Filenames are arbitrary and are not relied upon - each file's true type is determined internally.

## 3. Database Design

The Validation Service does not write to the database. IPMS persists the service's response and manages history. This section defines the tables IPMS must create.

### Tables required

| Table               | Purpose                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------|
| indent_analysis     | Stores every analysis run - score plus full detailed analysis; history with a latest flag |
| indent_feedback     | Stores user / manager feedback on findings, used offline to improve the service           |
| validation_standard | Stores the versioned best-practice standard per domain, read by the service at startup    |

### 3.1 indent\_analysis

Stores the score and the full detailed analysis. History is preserved by keeping all rows; the latest per indent is flagged for display.

| Column                   | Type          | Notes                                        |
|--------------------------|---------------|----------------------------------------------|
| analysis_id              | string        | Primary key                                  |
| indent_id                | string        | Indexed; links to the indent                 |
| domain                   | string        | Indent domain                                |
| analysis_timestamp       | datetime      | When analysis ran                            |
| score                    | number        | 0-100                                        |
| grade                    | string        | Strong / Adequate / Needs Improvement / Weak |
| score_breakdown          | object (JSON) | Per-category points                          |
| findings                 | list (JSON)   | Detailed findings                            |
| recommendations          | list (JSON)   | Recommended actions                          |
| missing_documents        | list (JSON)   | Required-but-absent document types           |
| interrelationship_issues | list (JSON)   | Cross-document inconsistencies               |
| document_inventory       | list (JSON)   | Classified documents + genuine flags         |
| standard_version         | string        | Which standard version was used              |
| is_latest                | boolean       | True for the most recent per indent          |
| full_response            | object (JSON) | Complete response, for audit                 |

**How “latest only” works.** On a new analysis for an indent, set `is_latest` = false on all prior rows for that `indent_id`, then insert the new row with `is_latest` = true. Display filters on `is_latest` = true; history returns all rows by timestamp.

### 3.2 indent\_feedback

Captures user / manager feedback on findings - read offline to improve prompts and correct the standard. Each row links to an analysis.

| Column      | Type   | Notes                    |
|-------------|--------|--------------------------|
| feedback_id | string | Primary key              |
| analysis_id | string | Links to indent_analysis |
| indent_id   | string | Indexed                  |
| domain      | string | Indent domain            |

| Column             | Type     | Notes                                                              |
|--------------------|----------|--------------------------------------------------------------------|
| feedback_timestamp | datetime | When feedback was given                                            |
| feedback_type      | string   | finding_wrong / finding_correct / severity_wrong / missing_finding |
| finding_category   | string   | mandatory / documentation / risk / vendor / approval               |
| finding_title      | string   | The finding being commented on                                     |
| user_comment       | text     | Free-text explanation                                              |
| correct_value      | text     | What the user says is correct (optional)                           |
| submitted_by       | string   | User or manager identifier                                         |

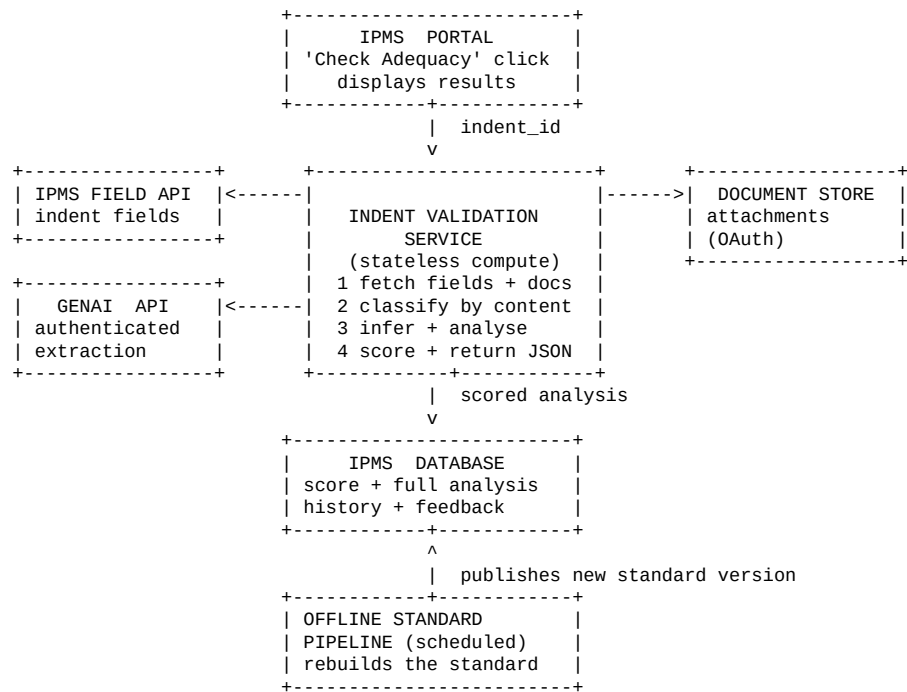
### 3.3 validation\_standard

Stores the versioned best-practice standard per domain. The service loads the current version into memory at startup and reloads when a new version is published.

| Column              | Type          | Notes                                  |
|---------------------|---------------|----------------------------------------|
| standard_id         | string        | Primary key                            |
| domain              | string        | Indent domain this standard applies to |
| version             | string        | Version label (e.g. v4-2026-06)        |
| published_timestamp | datetime      | When this version was published        |
| standard_body       | object (JSON) | The full standard content              |
| is_current          | boolean       | True for the active version per domain |

## 4. High-Level Architecture

The diagram below shows the systems involved and how they connect. The Validation Service is stateless except for the in-memory standard. Solid flow is runtime; the offline pipeline publishes the standard separately.



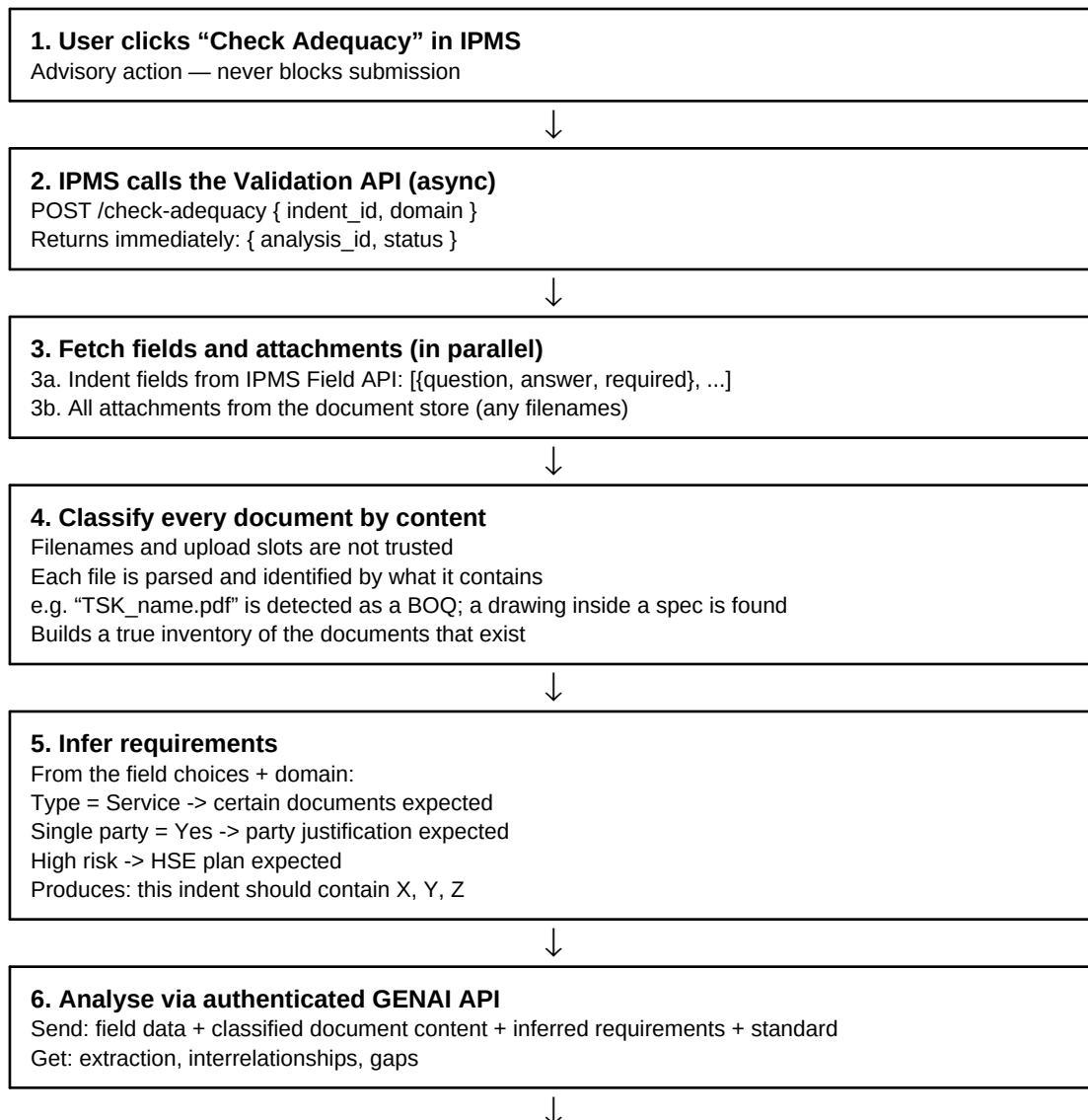
Four systems participate:

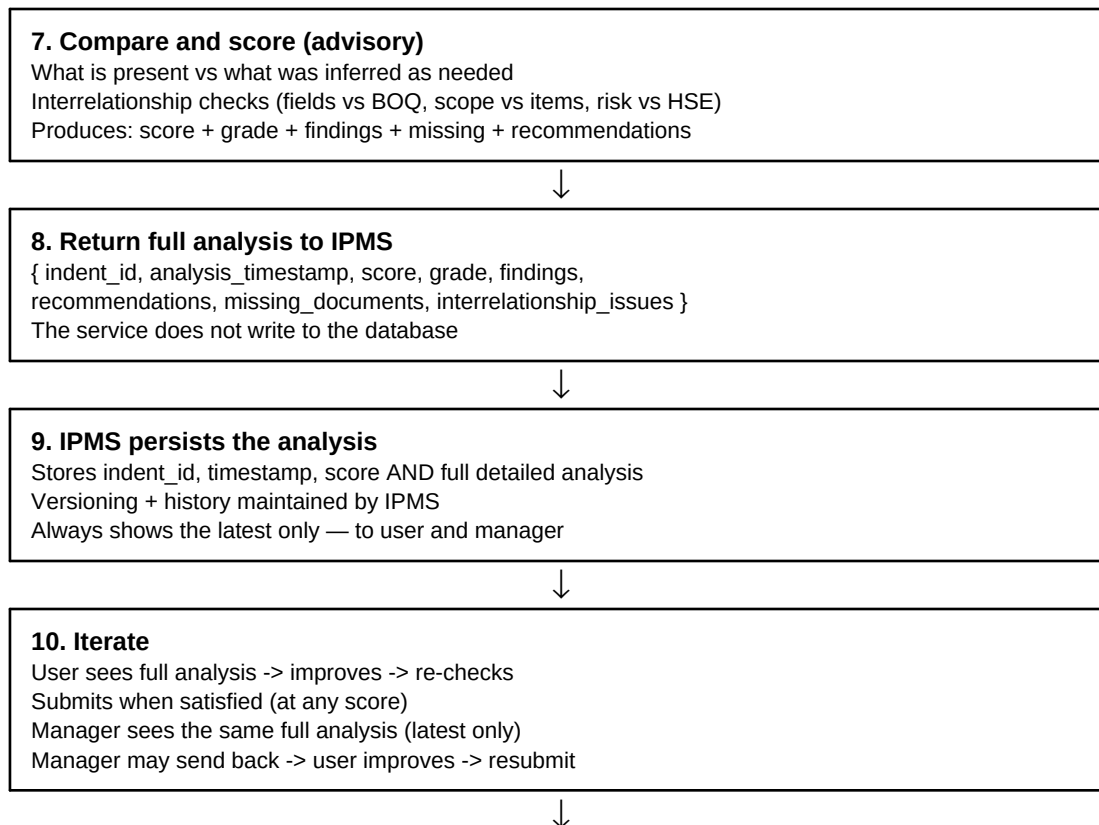
| System         | Role                                                   | Integration                                                  |
|----------------|--------------------------------------------------------|--------------------------------------------------------------|
| IPMS Portal    | Triggers analysis, displays results, owns the database | Calls the Check-Adequacy API; persists the response          |
| IPMS Field API | Source of indent field data                            | Service fetches question / answer / required entries         |
| Document Store | Source of attachments                                  | Service fetches files via authenticated OAuth                |
| GENAI API      | Extraction and analysis                                | Service sends prepared content over an authenticated session |

**Statelessness.** The service holds no persistent indent state. Each request is self-contained: fetch, analyse, return, forget. The only in-memory state is the current standard, loaded at startup and refreshed on a new version.

## 5. Data Flow - “Check Adequacy”

This is the runtime path, triggered each time a user selects “Check Adequacy”. It is asynchronous: the service acknowledges the request immediately and delivers the result when analysis completes.





## 5.1 Internal Analysis Steps (Summary)

Steps 4 and 5 are performed inside the service and are not integration concerns. In brief:

- **Classify documents by content** - attachments are identified by what they contain, not by filename or upload slot, and verified as genuine.
- **Infer requirements** - the service derives what the indent should contain from the field selections, then checks the actual content against that.

## 6. Standard Storage & Refresh

The best-practice standard is the benchmark every indent is measured against. It is stored in the `validation_standard` table and its lifecycle is separated from live traffic.

- Stored as a versioned record per domain (`validation_standard` table).
- Small and changes rarely.
- The live service loads the current version into memory at startup.
- The service periodically checks for a newer version and reloads if found - a rebuild takes effect without redeployment.

**No per-request database access for the standard.** Because the standard lives in memory, comparison is instant. The database is touched only at startup and on version change - never on the critical path of a request.

The offline pipeline rebuilds the standard on a schedule (a hybrid trigger: rebuild when 15 new quality-passed indents accumulate, or 30 days pass, whichever is first), then publishes a new version to the `validation_standard` table. Each analysis records the `standard_version` it used, so past scores remain reproducible.